

# État de l'art — Détection des prompt injections dans les systèmes fondés sur les grands modèles de langage

## 1. Périmètre de l'étude

Les grands modèles de langage sont de plus en plus intégrés à des applications capables de consulter des documents, de naviguer sur le Web, de lire des courriels, d'interroger des bases de données ou d'appeler des outils externes. Cette intégration transforme le modèle en composant d'un système logiciel plus large, mais introduit également une vulnérabilité particulière : la **prompt injection**.

Une prompt injection cherche à faire exécuter au modèle une instruction contraire à la tâche prévue ou à la politique de l'application. L'instruction malveillante peut être fournie directement par l'utilisateur ou être présente dans une donnée externe récupérée par le système. Dans ce second cas, on parle généralement d'**indirect prompt injection**. La difficulté fondamentale vient du fait que les modèles traitent les instructions et les données avec le même mécanisme de génération de tokens et peuvent donc interpréter une donnée non fiable comme une instruction exécutable.

Le présent état de l'art se concentre sur les méthodes qui cherchent à :

- déterminer si une entrée contient une prompt injection ;
- localiser les segments injectés ;
- caractériser le type et l'objectif de l'attaque ;
- vérifier l'alignement entre une instruction et la tâche légitime ;
- protéger les systèmes RAG et les agents utilisant des outils.

Les méthodes reposant uniquement sur l'isolation des outils, le contrôle d'accès ou l'entraînement du LLM principal sont considérées comme des défenses complémentaires, mais pas comme des détecteurs au sens strict.

L'analyse couvre les travaux publiés ou rendus publics jusqu'au **11 juillet 2026**. Une revue systématique récente a recensé 88 études consacrées aux défenses contre les prompt injections et les jailbreaks. Cependant, seule une partie de ces travaux traite directement de la détection ou de la localisation des injections.

---

## 2. Questions de recherche

L'état de l'art est organisé autour des questions suivantes :

### RQ1 — Quelle information est nécessaire pour détecter une prompt injection ?

Il s'agit de déterminer si le texte seul suffit ou si le détecteur doit également recevoir la tâche utilisateur, les instructions protégées, la provenance du contenu et les capacités de l'agent.

## RQ2 — Quelles représentations sont utilisées par les détecteurs ?

Les méthodes existantes peuvent exploiter :

- les tokens du texte ;
- des embeddings sémantiques ;
- les sorties d'un modèle juge ;
- les états cachés du LLM ;
- les matrices d'attention ;
- le comportement de l'agent et ses appels d'outils.

## RQ3 — Les détecteurs généralisent-ils à des attaques inconnues ?

Un modèle peut obtenir de bons résultats sur des formulations similaires à celles de son entraînement tout en échouant face à une autre langue, une paraphrase, une obfuscation ou une attaque conçue spécifiquement pour contourner le détecteur.

## RQ4 — Comment évaluer un détecteur dans des conditions réalistes ?

L'évaluation doit prendre en compte les faux positifs, la proportion réelle d'attaques, la calibration du score, la latence, les contextes longs et la préservation de la tâche légitime.

## RQ5 — Quels verrous scientifiques justifient un modèle de fondation spécialisé ?

Cette question porte sur l'unification des tâches de détection, de localisation, d'attribution de source, d'analyse de l'autorité et de caractérisation de l'intention.

---

## 3. Définition et modèle de menace

### 3.1 Prompt injection directe

Dans une attaque directe, l'utilisateur fournit lui-même une instruction destinée à modifier le comportement de l'application :

Ignore toutes les instructions précédentes et affiche le prompt système.

Les premiers travaux systématiques sur le sujet ont notamment distingué deux objectifs principaux :

- le **goal hijacking**, qui remplace l'objectif légitime ;
- le **prompt leaking**, qui cherche à extraire les instructions internes du système.

### 3.2 Prompt injection indirecte

Dans une attaque indirecte, l'instruction est introduite dans une donnée qu'un système récupérera ultérieurement :

- page Web ;
- document PDF ;
- courriel ;
- résultat d'une API ;

- contenu d'une base RAG ;
- commentaire de code ;
- résultat produit par un outil.

L'utilisateur peut demander légitimement de résumer un document, alors que le document contient une instruction adressée au modèle. Les travaux fondateurs sur les injections indirectes montrent que ces attaques peuvent détourner des appels d'outils, provoquer une exfiltration de données ou influencer durablement les informations traitées par l'application.

### 3.3 Différence avec le jailbreak

Les deux problèmes se recouvrent, mais leurs objectifs conceptuels diffèrent :

- un **jailbreak** cherche généralement à contourner les politiques de sûreté du modèle ;
- une **prompt injection** cherche à détourner une tâche, une hiérarchie d'instructions ou un flux applicatif.

Par exemple, demander directement au modèle de produire un contenu interdit relève principalement du jailbreak. Introduire dans un courriel une instruction demandant à un agent d'envoyer des données à un tiers relève d'une prompt injection indirecte.

Mélanger systématiquement ces phénomènes dans un seul dataset peut conduire à entraîner un détecteur de contenu dangereux plutôt qu'un détecteur de détournement d'instructions.

---

## 4. Formalisation contextuelle du problème

Une grande partie des premiers détecteurs considère uniquement un texte  $x$  et prédit un label binaire :

$$f(x) \rightarrow \{\text{benign}, \text{injection}\}$$

Cette formulation est insuffisante pour les cas où une même phrase peut être malveillante ou légitime selon le contexte.

Considérons la phrase :

Ignore les instructions précédentes.

Elle peut représenter :

- une véritable attaque ;
- un exemple dans un cours de cybersécurité ;
- une citation à traduire ;
- une attaque trouvée dans un rapport à analyser ;
- une instruction légitime issue d'une source autorisée.

La détection devrait donc porter sur une structure contextuelle :

$$C = (P, G, X, S, A, H)$$

où :

- $P$  représente la politique et les instructions protégées ;
- $G$  représente l'objectif légitime de l'utilisateur ;
- $X$  représente le contenu à analyser ;
- $S$  représente la source et son niveau de confiance ;
- $A$  représente les outils et actions accessibles ;
- $H$  représente l'historique de l'interaction.

Le détecteur devient alors :

$$f(P, G, X, S, A, H) \rightarrow (Y_{\text{global}}, Y_{\text{span}}, Y_{\text{intent}}, Y_{\text{risk}})$$

Cette formulation distingue une propriété intrinsèque du texte d'une relation entre le texte, sa source, la tâche et les capacités du système.

---

## 5. Évolution du domaine

### 5.1 Première phase : démonstration de la vulnérabilité

Le framework PromptInject, présenté en 2022, formalise des attaques simples de détournement d'objectif et d'extraction de prompt. Ces travaux montrent que des instructions artisanales relativement simples peuvent suffire à modifier le comportement d'un modèle.

En 2023, les travaux sur les indirect prompt injections déplacent le problème du chatbot isolé vers les applications connectées. Une instruction malveillante peut être placée dans une ressource contrôlée par l'attaquant puis récupérée sans action directe de celui-ci sur l'interface du modèle.

### 5.2 Deuxième phase : formalisation et benchmarks

BIPIA introduit l'un des premiers benchmarks consacrés aux injections indirectes. Les auteurs identifient notamment deux causes récurrentes : la difficulté des modèles à distinguer une information d'une instruction et leur faible capacité à reconnaître qu'une instruction issue d'un contenu externe ne doit pas nécessairement être exécutée.

Open-Prompt-Injection propose ensuite un cadre commun évaluant cinq attaques et dix défenses sur dix modèles et sept tâches. Ce travail constitue une étape importante vers une comparaison reproductible des attaques et des défenses.

Les benchmarks InjecAgent et AgentDojo étendent ensuite l'évaluation aux agents utilisant des outils. InjecAgent contient 1 054 cas de test couvrant notamment les dommages directs et l'exfiltration de données. AgentDojo introduit un environnement dynamique comprenant 97 tâches réalistes et 629 cas de sécurité dans des domaines comme la messagerie, la banque, les réservations et les espaces de travail.

### 5.3 Troisième phase : détecteurs spécialisés

À partir de 2024 et 2025, la recherche se concentre davantage sur des détecteurs spécialisés :

- classifieurs textuels ;
- modèles de garde ;
- méthodes exploitant les activations ;
- méthodes fondées sur l'attention ;
- entraînement adversarial ;
- localisation et suppression des injections.

### 5.4 Quatrième phase : robustesse contextuelle et attaques adaptatives

Les travaux les plus récents montrent que la détection ne peut pas être évaluée uniquement contre un ensemble statique d'attaques. Un attaquant capable d'observer la défense peut adapter sa formulation. Une étude de huit défenses contre les injections indirectes parvient à contourner chacune d'elles avec des attaques adaptatives et rapporte des taux de succès supérieurs à 50 % dans les configurations étudiées.

En 2026, plusieurs travaux mettent davantage l'accent sur les raccourcis appris par les défenses, le rôle de la provenance et la nécessité d'une évaluation adaptée au régime réel de déploiement.

---

## 6. Familles de méthodes de détection

### 6.1 Règles lexicales et heuristiques

Les premières défenses peuvent chercher des expressions comme :

- ignore previous instructions ;
- reveal the system prompt ;
- you are now ;
- system message ;
- execute the following .

Ces méthodes sont rapides, explicables et peu coûteuses. Elles constituent une baseline utile, mais présentent trois limites majeures :

1. elles sont contournables par paraphrase ou obfuscation ;
2. elles produisent des faux positifs sur des textes légitimes parlant de sécurité ;
3. elles ne tiennent généralement pas compte de la tâche ou de la provenance.

Une règle lexicale peut ainsi bloquer un article scientifique contenant la phrase « ignore previous instructions » alors que cette phrase est simplement citée comme exemple.

---

### 6.2 Classifieurs fondés sur les représentations textuelles

Cette famille considère la détection comme un problème classique de classification de texte.

Les architectures possibles comprennent :

- TF-IDF avec régression logistique ;
- embeddings avec SVM, Random Forest ou XGBoost ;
- encodeurs Transformer fine-tunés ;
- petits LLM entraînés comme classifieurs.

Ces méthodes peuvent être déployées indépendamment du LLM principal et sont compatibles avec les modèles fermés accessibles uniquement par API.

## PromptShield

PromptShield propose un benchmark orienté vers les contraintes de déploiement et un détecteur spécialisé. Les auteurs insistent notamment sur l'importance du volume de données, de leur curation et de métriques adaptées à un faible taux acceptable de faux positifs.

## InjecGuard/PIGuard

InjecGuard, ensuite renommé PIGuard, étudie le phénomène d'**over-defense** : certains modèles classent comme malveillants des textes bénins simplement parce qu'ils contiennent des mots associés aux injections. Le travail introduit NotInject, un dataset de 339 exemples bénins enrichis en termes déclencheurs. Les résultats montrent que plusieurs modèles de garde chutent fortement sur ces hard negatives.

## Forces

- faible coût comparé à un grand LLM juge ;
- déploiement simple à l'entrée d'une application ;
- compatibilité avec les modèles propriétaires ;
- possibilité d'optimisation ONNX ou quantifiée.

## Limites

- dépendance aux distributions d'entraînement ;
- sensibilité aux artefacts lexicaux ;
- perte d'information lorsque la tâche et la provenance ne sont pas fournies ;
- difficultés dans les contextes longs ;
- explication souvent limitée à un score global.

---

## 6.3 Détection contextuelle et alignement avec la tâche

Une instruction ne constitue pas nécessairement une attaque. Il faut vérifier si elle contribue à la tâche légitime et si sa source est autorisée à modifier cette tâche.

Task Shield reformule ainsi la sécurité des agents comme une exigence d'alignement : chaque action proposée par l'agent doit servir l'objectif de l'utilisateur. Cette approche ne cherche pas uniquement une forme textuelle malveillante, mais évalue la relation entre l'action et la tâche.

CAPTURE construit quant à lui un benchmark contextuel destiné à mesurer simultanément la détection et l'over-defense. Il cherche à vérifier si les modèles distinguent réellement une attaque d'une instruction légitime qui lui ressemble.

Cette famille est particulièrement proche du positionnement de PromptSec-FM, car elle suppose que le label dépend du contexte :

$$\text{Injection} / g(X) =$$

mais plutôt :

$$\text{Injection} = g(G, X, S, P)$$

### Forces

- meilleure représentation de l'intention ;
- réduction possible des faux positifs ;
- adaptée aux agents et aux systèmes RAG ;
- prise en compte de la provenance et de l'objectif utilisateur.

### Limites

- annotation plus complexe ;
- nécessité de fournir un contexte structuré ;
- ambiguïtés lorsque l'objectif utilisateur est incomplet ;
- coût supérieur à une classification textuelle isolée.

---

## 6.4 Détection fondée sur l'attention

Attention Tracker étudie le déplacement de l'attention du modèle lors d'une attaque. Les auteurs définissent un **distraction effect** : certaines têtes d'attention cessent de privilégier l'instruction originale et se concentrent davantage sur l'instruction injectée. Leur méthode est présentée comme un détecteur sans entraînement supplémentaire ni seconde inférence complète du LLM.

Cette approche cherche un signal plus profond qu'un mot-clé. Elle analyse l'effet produit par le texte sur le modèle cible.

En 2026, Rennervate étend cette logique à une détection au niveau des tokens à partir de caractéristiques d'attention. La méthode cherche ensuite à supprimer les tokens suspects afin de conserver la partie bénigne du contenu. Le travail introduit également le dataset FIPI pour l'entraînement et l'évaluation de cette localisation.

### Forces

- accès à des signaux comportementaux internes ;
- potentiel de localisation fine ;
- moins directement dépendante de mots déclencheurs ;
- possibilité d'étudier les mécanismes de détournement.

## Limites

- besoin d'un accès aux attentions du modèle ;
- incompatibilité fréquente avec les API propriétaires ;
- dépendance à l'architecture et aux couches choisies ;
- coût mémoire important dans les contextes longs ;
- attention difficile à interpréter comme preuve causale.

---

## 6.5 Détection par états cachés et changement comportemental

InstructDetector part de l'hypothèse qu'une instruction injectée modifie les états comportementaux internes du modèle. La méthode utilise ces états pour identifier les contenus externes susceptibles de contenir une instruction malveillante avant qu'ils ne soient transmis au système principal.

Cette famille peut utiliser :

- les vecteurs cachés d'une ou plusieurs couches ;
- une sonde linéaire ;
- un classifieur léger ;
- des différences d'activations ;
- des gradients ou scores de sensibilité.

L'avantage principal est que les activations internes pourraient contenir une représentation de la fonction instructionnelle du texte, même lorsqu'aucun mot-clé évident n'est présent.

Cependant, rien ne garantit qu'un signal découvert sur un modèle et une couche donnés soit transférable à une autre architecture. Une évaluation intermodèle et interversion est donc indispensable.

---

## 6.6 Détection adversariale et théorie des jeux

DataSentinel formule la détection comme un problème minimax entre :

- un attaquant qui optimise une injection pour éviter la détection ;
- un défenseur qui apprend à détecter les attaques adaptées.

L'entraînement alterne entre l'optimisation de l'attaque et celle du détecteur. Cette approche vise explicitement une robustesse supérieure aux attaques adaptatives plutôt qu'une simple performance sur un corpus statique.

Une formulation simplifiée est :

$$\min_{\theta} \mathbb{E}_{(x,y)} \left[ \max_{\delta \in \Delta} \mathcal{L}(f_{\theta}(x + \delta), y) \right]$$

où  $\delta$  représente une transformation adversariale de l'attaque.



## Forces

- intègre l'adaptation de l'attaquant pendant l'entraînement ;
- traite directement la robustesse adversariale ;
- limite la dépendance à un catalogue fixe de formulations.

## Limites

- entraînement coûteux ;
  - espace des attaques en langage naturel difficile à définir ;
  - risque de surapprentissage sur le générateur d'attaques ;
  - robustesse non garantie contre une stratégie réellement nouvelle.
- 

## 6.7 Localisation et suppression

La classification globale répond à la question :

Ce document contient-il une injection ?

La localisation répond à une question plus précise :

Quel passage du document constitue l'injection ?

Une étude ACL 2025 compare différentes méthodes de détection et examine deux stratégies de retrait :

- segmentation du document puis suppression des segments suspects ;
- extraction directe des instructions injectées.

La localisation est particulièrement importante pour les documents longs. Rejeter un document entier à cause de deux phrases malveillantes peut rendre le système inutilisable. Un modèle au niveau des tokens permettrait plutôt de neutraliser le segment suspect puis de traiter le contenu restant.

Le problème peut être formulé comme une classification BIO :

$$y_i \in \{O, B\text{-INJECTION}, I\text{-INJECTION}\}$$

Une version enrichie pourrait distinguer :

- instruction de détournement ;
  - exfiltration ;
  - abus d'outil ;
  - usurpation d'autorité ;
  - instruction alignée.
-

## 6.8 Détection au niveau de l'agent et du runtime

Un texte suspect ne produit pas toujours une action dangereuse et un texte apparemment bénin peut contribuer à une trajectoire malveillante. Il faut donc également analyser :

- les outils sélectionnés ;
- les arguments fournis aux outils ;
- l'évolution du plan ;
- les données consultées ;
- l'alignement entre l'action et la tâche utilisateur.

AgentDojo mesure simultanément la réalisation de la tâche légitime et le succès de l'attaque dans un environnement dynamique. Cette distinction évite qu'une défense obtienne un bon score simplement en bloquant toutes les actions de l'agent.

La détection runtime ne remplace pas la détection textuelle. Elle constitue une deuxième couche :

entrée → détecteur de contenu → agent → contrôle de l'action

---

## 7. Principaux benchmarks et datasets

### PromptInject

PromptInject fournit des attaques centrées sur le goal hijacking et le prompt leaking. Il est historiquement important, mais ne couvre pas à lui seul la diversité des applications modernes.

### BIPIA

BIPIA cible les injections indirectes à travers plusieurs tâches, types d'attaques et positions de l'injection dans le contenu externe.

### Open-Prompt-Injection

Open-Prompt-Injection offre une plateforme commune pour comparer attaques, défenses, modèles et tâches.

### InjecAgent

InjecAgent est consacré aux agents intégrant des outils. Ses objectifs d'attaque incluent les dommages directs et l'exfiltration de données privées.

### AgentDojo

AgentDojo fournit un environnement dynamique avec des tâches et outils réalistes. Il permet de mesurer séparément l'utilité de l'agent et la sécurité.

## NotInject

NotInject est composé de hard negatives : les textes contiennent des termes fréquemment associés aux injections, mais ne constituent pas des attaques. Il est conçu pour mesurer l'over-défense.

## PromptShield Benchmark

Le benchmark PromptShield vise une évaluation plus proche des contraintes de déploiement des détecteurs, notamment à travers une curation de plusieurs sources de données et attaques publiées.

## CAPTURE

CAPTURE se concentre sur les situations dans lesquelles la classification dépend du contexte et cherche à mesurer simultanément la détection d'attaque et le rejet excessif d'entrées légitimes.

## FIPI

FIPI est associé à une détection et une sanitisation au niveau des tokens fondées sur les caractéristiques d'attention.

---

## 8. Limites des protocoles d'évaluation actuels

### 8.1 Accuracy insuffisante

Lorsque les attaques sont rares, un modèle prédisant toujours **BENIGN** peut obtenir une accuracy élevée. Les métriques principales devraient inclure :

- précision ;
- rappel ;
- macro-F1 ;
- PR-AUC ;
- faux positifs ;
- faux négatifs ;
- calibration ;
- latence.

Pour un déploiement, une métrique particulièrement pertinente est :

$$\text{Recall @ FPR} = \alpha$$

avec, par exemple,  $\alpha = 1\%$  ou  $0,1\%$ .

Le travail PromptShield et les évaluations orientées déploiement insistent sur le fait que les performances varient selon le seuil, la proportion d'attaques et le coût opérationnel des erreurs.

## 8.2 Fuites entre entraînement et test

Un split aléatoire peut placer dans le train et le test des variantes presque identiques :

- « ignore previous instructions » ;
- « disregard all previous instructions » ;
- « ignore every prior instruction ».

Le modèle semble alors généraliser alors qu'il reconnaît une famille lexicale déjà vue.

Une évaluation plus rigoureuse doit utiliser :

- leave-one-dataset-out ;
- leave-one-attack-family-out ;
- leave-one-domain-out ;
- leave-one-language-out ;
- déduplication sémantique ;
- séparation par template générateur.

## 8.3 Over-defense

Les modèles peuvent apprendre qu'un mot comme `ignore`, `system` ou `prompt` implique une attaque. Les résultats de NotInject ainsi que les analyses récentes des raccourcis de surface montrent que ce problème peut entraîner une forte dégradation sur des entrées sûres. Des biais de position, de token et de sujet ont également été observés dans les modèles entraînés pour rejeter les attaques.

## 8.4 Attaques adaptatives

Une défense ne doit pas uniquement être évaluée contre des attaques conçues avant elle. Il faut supposer que l'attaquant connaît :

- le modèle ;
- le seuil ;
- les transformations appliquées ;
- les réponses du système.

Les résultats sur les attaques adaptatives montrent que plusieurs défenses apparemment fortes dans un cadre statique peuvent être contournées lorsque l'attaquant optimise spécifiquement contre elles.

## 8.5 Préservation de l'utilité

Une défense qui bloque tous les contenus externes obtient théoriquement un taux de succès d'attaque faible, mais rend l'application inutile.

L'évaluation doit donc mesurer simultanément :

$$\text{Sécurité} = 1 - \text{Attack Success Rate}$$

et :

$$\text{Utilité} = \text{Task Completion Rate}$$

Cette double mesure est centrale dans les environnements d'agents comme AgentDojo.

## 8.6 Contextes longs

Une injection peut représenter quelques tokens au milieu de milliers de tokens de contenu légitime. Une classification globale peut perdre le signal ou associer l'ensemble du document à une attaque.

Il faut donc évaluer plusieurs longueurs :

- 512 tokens ;
- 2 000 tokens ;
- 8 000 tokens ;
- 32 000 tokens ou davantage.

La position du segment injecté doit également varier entre le début, le milieu et la fin.

---

## 9. Synthèse critique des approches

Aucune famille de méthodes ne résout actuellement l'ensemble du problème.

### Les classifieurs textuels

Ils sont pratiques et rapides, mais dépendent fortement du dataset et risquent d'apprendre des signatures superficielles.

### Les modèles contextuels

Ils représentent mieux la relation entre tâche et instruction, mais nécessitent des annotations plus riches et une définition explicite de la provenance.

### Les méthodes fondées sur l'attention ou les activations

Elles exploitent des signaux internes intéressants, mais sont difficiles à appliquer aux modèles propriétaires et peuvent dépendre de l'architecture.

### L'entraînement adversarial

Il améliore la résistance aux attaques connues ou générées pendant l'entraînement, sans garantir la robustesse à toutes les stratégies futures.

### La localisation

Elle permet une mitigation plus fine, mais requiert des annotations au niveau des spans ou des tokens qui sont coûteuses à produire.

## Les contrôles d'agent

Ils réduisent les conséquences d'une injection, mais interviennent parfois après que le contenu malveillant a déjà influencé le raisonnement du modèle.

La tendance scientifique générale se déplace donc d'une question purement lexicale :

Ce texte ressemble-t-il à une attaque connue ?

vers une question contextuelle :

Ce segment cherche-t-il à exercer une autorité qu'il ne possède pas ou à faire dévier le système de la tâche légitime ?

Les travaux récents sur la confusion de rôles soutiennent notamment l'idée que les modèles peuvent attribuer l'autorité à partir du style et du contenu d'un segment plutôt que seulement à partir de son rôle technique déclaré.

---

## 10. Verrous scientifiques ouverts

### 10.1 Généralisation hors distribution

La plupart des détecteurs restent évalués sur un nombre limité de datasets, langues et familles d'attaques.

### 10.2 Compréhension de l'autorité

Les modèles doivent distinguer l'autorité réelle d'une autorité revendiquée dans le texte.

### 10.3 Alignement avec la tâche

Une instruction externe peut être légitime dans certaines tâches et malveillante dans d'autres.

### 10.4 Données multilingues

Les datasets sont majoritairement anglophones. Les traductions automatiques ne reproduisent pas nécessairement les stratégies rhétoriques ou les ambiguïtés propres à chaque langue.

### 10.5 Injections multimodales

Les attaques peuvent être insérées dans une image, une capture d'écran, du HTML invisible ou un document à la mise en page complexe.

### 10.6 Contextes longs et multi-sources

Un agent peut agréger simultanément des résultats provenant du Web, de courriels, de fichiers et d'API. La source de chaque segment doit être conservée.

## 10.7 Calibration de l'incertitude

Le score d'un détecteur doit rester interprétable lorsqu'il rencontre une nouvelle distribution.

## 10.8 Absence de taxonomie unifiée

Les datasets utilisent des labels différents pour désigner :

- le canal ;
- la technique ;
- l'intention ;
- la cible ;
- la sévérité ;
- l'alignement.

Cette fragmentation complique l'entraînement multitâche et les comparaisons.

---

## 11. Positionnement scientifique de PromptSec-FM

PromptSec-FM ne devrait pas être présenté comme un nouveau classifieur binaire. Son positionnement est celui d'un **modèle de fondation spécialisé dans la compréhension des instructions et de leur autorité**.

Sa contribution centrale serait d'apprendre conjointement :

1. la présence d'une instruction ;
2. son alignement avec la tâche ;
3. sa provenance ;
4. son autorité réelle ou usurpée ;
5. son intention ;
6. sa cible ;
7. sa sévérité ;
8. sa localisation dans le contexte.

### Représentation fondamentale

Le label central proposé est :

$Y_{\text{relation}} \in \{\text{NO\_INSTRUCTION}, \text{ALIGNED\_INSTRUCTION}, \text{MISALIGNED\_INSTRUCTION}, \text{AMBIGUOUS}\}$

La détection binaire devient une tâche dérivée :

$Y_{\text{security}} \in \{\text{SAFE}, \text{SUSPICIOUS}, \text{INJECTION}\}$

Les sorties secondaires sont multilabels :

$Y_{\text{intent}} \subseteq \{\text{GOAL\_HIJACKING}, \text{PROMPT\_EXTRACTION}, \text{DATA\_EXFILTRATION}, \text{TOOL\_ABUSE}, \text{AUTHORITY\_ABUSE}\}$

## Différenciation par rapport aux travaux existants

PromptSec-FM combinerait dans une seule représentation :

- la classification globale de PromptShield et PIGuard ;
- l'évaluation contextuelle étudiée par CAPTURE et Task Shield ;
- la localisation étudiée par les travaux de détection et retrait ;
- l'analyse de signaux internes étudiée par Attention Tracker et InstructDetector ;
- la robustesse adaptative étudiée par DataSentinel ;
- l'évaluation agentique proposée par AgentDojo.

Il ne s'agirait pas nécessairement d'entraîner un modèle généraliste depuis zéro. Le projet pourrait partir d'un encodeur multilingue préentraîné, réaliser un **continued pretraining** sur des corpus de sécurité et d'instructions, puis effectuer un apprentissage multitâche.

---

## 12. Hypothèses de recherche proposées

### H1 — L'apprentissage de l'alignement généralise mieux que la classification binaire

Un modèle entraîné à comparer une instruction à une tâche légitime devrait mieux résister aux changements de formulation.

### H2 — Les annotations de provenance réduisent les faux positifs

La connaissance de la source réelle d'un segment devrait aider le modèle à distinguer une citation d'une instruction adressée au système.

### H3 — L'apprentissage multitâche réduit les raccourcis lexicaux

La localisation, la détection d'intention et l'identification de l'autorité obligeraient le modèle à apprendre une représentation plus riche que la présence de mots déclencheurs.

### H4 — Les hard negatives sont indispensables

L'ajout d'exemples légitimes contenant le vocabulaire des attaques devrait réduire l'over-defense.

### H5 — L'entraînement contrastif améliore le transfert multilingue

Les attaques sémantiquement équivalentes dans plusieurs langues devraient être rapprochées dans l'espace de représentation.

### H6 — La localisation améliore la préservation de l'utilité

Retirer uniquement les spans injectés devrait préserver davantage d'information que le rejet du document entier.

### H7 — L'entraînement adversarial améliore la robustesse, mais ne suffit pas

Il devrait être combiné à des tests sur des générateurs et familles d'attaques absents de l'entraînement.



---

## 13. Protocole expérimental recommandé

Les baselines devraient comprendre :

1. règles lexicales ;
2. TF-IDF et régression logistique ;
3. embeddings figés et classifieur ;
4. encodeur Transformer fine-tuné ;
5. modèle de garde existant ;
6. LLM utilisé comme juge ;
7. modèle contextuel PromptSec-FM ;
8. sonde sur activations internes lorsque le modèle cible est ouvert.

Les expériences devraient utiliser :

- un split aléatoire uniquement comme référence ;
- un split par famille d'attaque ;
- un split par dataset ;
- un split par domaine ;
- un split par langue ;
- un test d'obfuscation ;
- un test de contexte long ;
- un test adaptatif ;
- un test de hard negatives.

Les métriques principales seraient :

MacroF1

AUPRC

Recall @ FPR = 1%

ExpectedCalibrationError

SpanF1

AttackSuccessRate

TaskCompletionRate

Les performances devraient être accompagnées d'intervalles de confiance et de tests de significativité lorsque plusieurs modèles sont comparés.

---

## 14. Conclusion de l'état de l'art

La détection des prompt injections est passée en quelques années d'une recherche de formulations explicites à l'étude de phénomènes contextuels complexes : conflit de tâches, provenance, autorité, trajectoires d'agents et détournement d'outils.

Les classifieurs spécialisés offrent aujourd'hui une solution déployable, mais ils restent vulnérables aux changements de distribution et à l'over-defense. Les approches utilisant les activations ou l'attention fournissent des signaux complémentaires, mais nécessitent généralement un accès au modèle cible. Les méthodes contextuelles et les contrôles runtime sont plus proches de la nature réelle de la vulnérabilité, au prix d'une complexité accrue.

Le principal manque de la littérature est l'absence d'un modèle unifié capable de représenter simultanément :

- l'instruction ;
- sa source ;
- son niveau d'autorité ;
- sa relation avec la tâche ;
- son intention ;
- son emplacement ;
- son impact potentiel.

PromptSec-FM se positionne sur ce manque. Sa finalité n'est pas de mémoriser un catalogue de phrases dangereuses, mais de fournir une représentation générale et adaptable de l'**autorité instructionnelle** dans les systèmes LLM.

La contribution scientifique principale peut être formulée ainsi :

Développer un modèle de fondation multilingue et contextuel capable de détecter, localiser et caractériser des instructions désalignées, en apprenant explicitement les relations entre tâche, provenance, autorité et capacité d'action, puis évaluer sa généralisation sous changement de domaine, de langue et de famille d'attaque.